First Lab Assignment: System Modeling and Profiling

STUDENTS IDENTIFICATION:

Number:	Name:
103363	Alexandre Shcherban Vukovod
103796	Tomás Sabral Teixeira
103580	Érik Stelvin da Silva Bianchi

2 Exercise

Please justify all your answers with values from the experiments.

1. What is the cache capacity of the computer you used (please write the workstation name)?

	Array Size	4K	8K	16K	32K	64K	128K
(ms)	t2-t1	0,891	1,753	3,407	6,8	16,87	90,35
X N_REPETITIONS	# accesses a[i]	819200	1638400	3276800	6553600	13107200	26214400
(ms)	# mean access time	2,18	2,15	2,08	2,08	2,58	3,08

The cache capacity of the PC (lab 6 p.7) is 32 KB because between the array with size 32K and 64K is where we see the biggest increase in the access time, wich means that the array is bigger than the cache.

Consider the data presented in Figure 1. Answer the following questions (2, 3, 4) about the machine used to generate that data.

2. What is the cache capacity?

The cache capacity is 64 KB because when we use arrays smaller than 64 KB we get low times and when we increase it above 64 KB, the time starts increasing.

3. What is the size of each cache block?

The size of each cache is 16 B (words) because if we look at the array bigger than 64KB, the graph stabilizes at a stride of 16 where it is always missing, meaning that it is accessing another block of the cache.

4. What is the L1 cache miss penalty time?

The cache miss penalty is 1000 − 370 = 630 ms (approximately).
It is the difference between the miss time (when the 128K and above arrays stabilize) and the hit time (where the 64K and bellow arrays stabilize)

3 Procedure

3.1.1 Modeling the L1 Data Cache

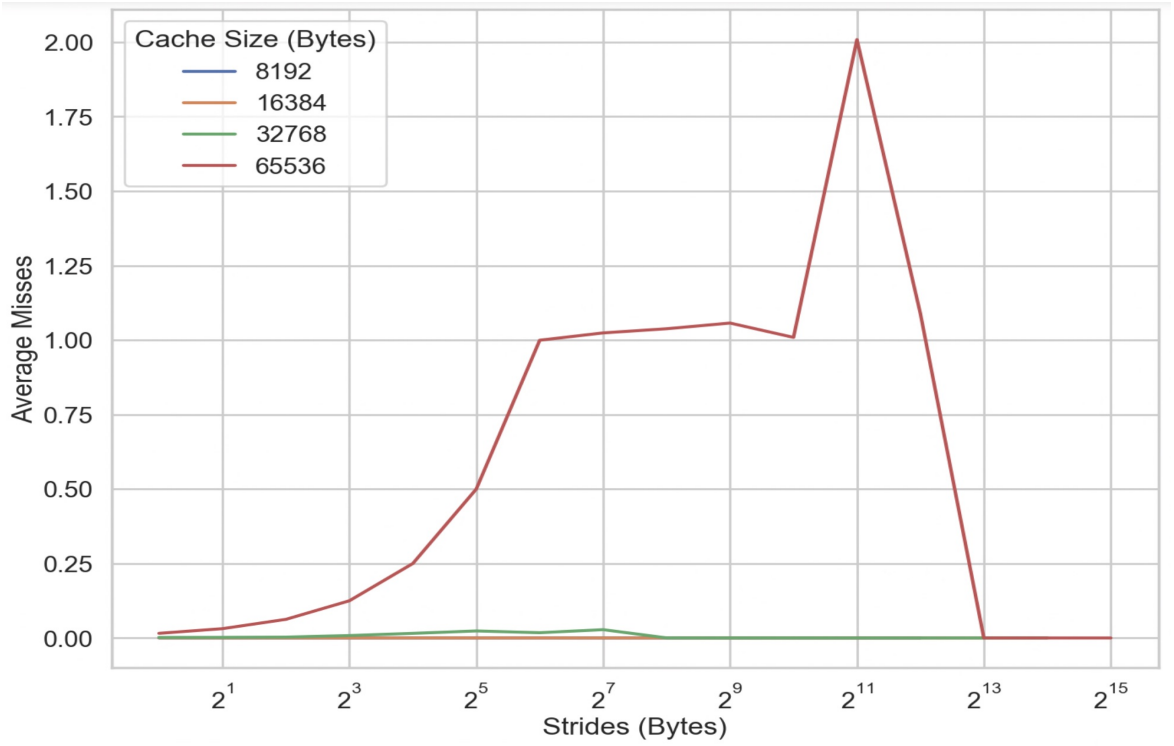
a) What are the processor events that will be analyzed during its execution? Explain their meaning.

The only event that is being analyzed is PAPI_L1_DCM (L1 Data Cache Misses). This event will analyze the number of L1 data cache misses during the execution of the code, that is, the number of times that the processor tried to get data from the L1 cache and it wasn't there.

b) Plot the variation of the average number of misses (*Avg Misses*) with the `stride` size, for each considered dimension of the L1 data cache (8kB, 16kB, 32kB and 64kB).
Note that, you may fill these tables and graphics (as well as the following ones in this report) on your computer and submit the printed version.

Array Size	Stride	Avg Misses	Avg Cycl Time
8kBytes	1	0,000186	0,002316
	2	0,000094	0,002251
	4	0,000051	0,002239
	8	0,000044	0,002178
	16	0,000044	0,002248
	32	0,000044	0,002264
	64	0,000046	0,002188
	128	0,000024	0,002057
	256	0,000017	0,001998
	512	0,000009	0,001985
	1024	0,000005	0,001951
	2048	0,000005	0,002014
	4096	0,000005	0,002078
16kBytes	1	0,000199	0,002244
	2	0,000136	0,002240
	4	0,000130	0,002227
	8	0,000149	0,002169
	16	0,000140	0,002238
	32	0,000231	0,002239
	64	0,000164	0,002196
	128	0,000089	0,002183
	256	0,000045	0,002058
	512	0,000015	0,002005
	1024	0,000010	0,001986
	2048	0,000008	0,002083
	4096	0,000004	0,002167
	8192	0,000003	0,002077

Array Size	Stride	Avg Misses	Avg Cycl Time
32kBytes	1	0,001886	0,002217
	2	0,002759	0,002205
	4	0,004145	0,002192
	8	0,007451	0,002178
	16	0,013011	0,002015
	32	0,017827	0,002233
	64	0,016861	0,002212
	128	0,059418	0,002209
	256	0,086654	0,002209
	512	0,000156	0,002060
	1024	0,000066	0,001998
	2048	0,000032	0,002047
	4096	0,000019	0,002189
	8192	0,000004	0,002173
	16384	0,000002	0,002078
64kBytes	1	0,015654	0,001976
	2	0,031307	0,001889
	4	0,062670	0,002092
	8	0,125341	0,002210
	16	0,250615	0,002154
	32	0,501081	0,002250
	64	1,000664	0,001857
	128	1,250062	0,001890
	256	1,040599	0,001928
	512	1,052760	0,001929
	1024	1,011275	0,001854
	2048	1,774964	0,004842
	4096	1,041282	0,002051
	8192	0,000014	0,002188
	16384	0,000001	0,002166
	32768	0,000001	0,002078



c) By analyzing the obtained results:

- Determine the **size** of the L1 data cache. Justify your answer.

The size of the L1 data cache is 32 KB because in the graphic above the biggest array that has almost zero average misses is the 32 K one, meaning that it fits in its entirety inside the cache.

- Determine the **block size** adopted in this cache. Justify your answer.

To determine the block size we have to look at the 64 K array in the graph because it is the only one where average misses vary. By looking at it we can determine that the block size is 2^6 bytes (64 B) because until that point the stride is small enough that we are getting some hits in the same block but above that point, when the plot flattens, we are always missing. This means that we are always accessing different blocks

- Characterize the **associativity set size** adopted in this cache. Justify your answer.

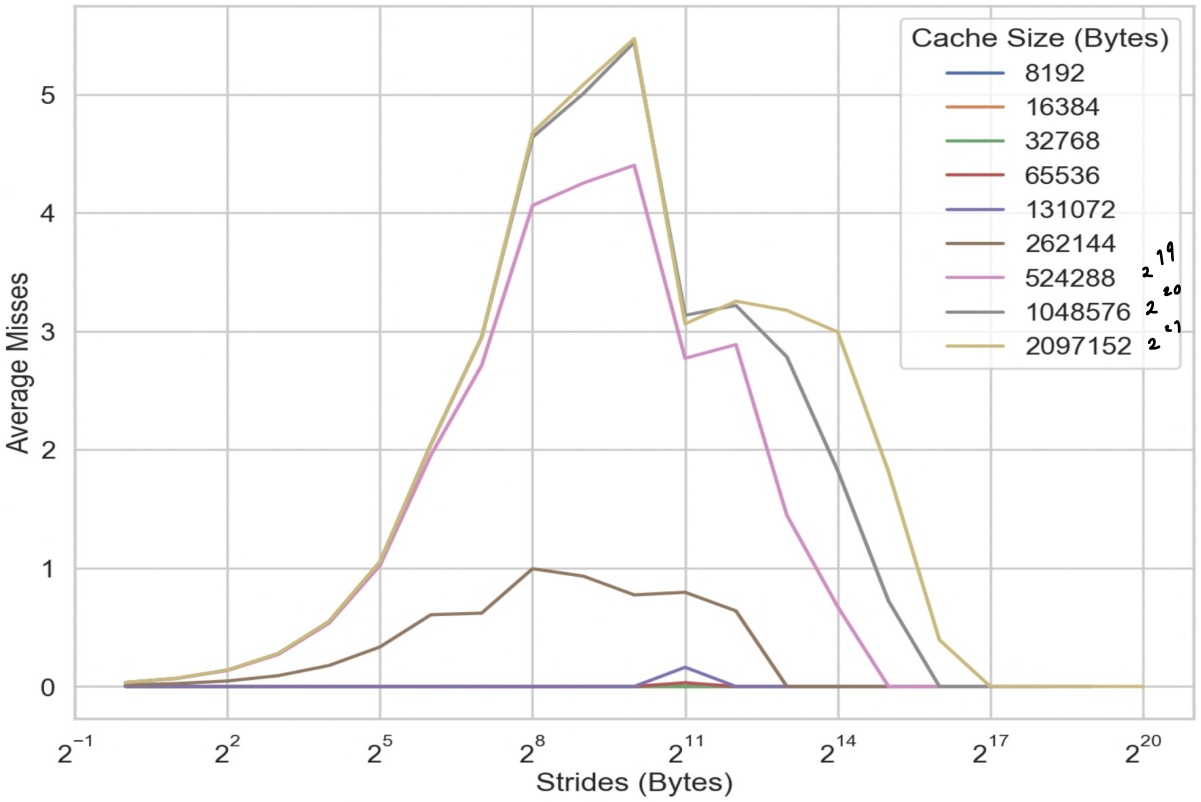
To calculate the associativity set size we have to look at the first cache size that is bigger than the actual L1 cache. In this case we will look at the 64K one (red plot line). If we look at it, we see a big drop in the average misses (it comes down to zero). Since we already know that our block size is 64K, we can conclude that this drop is caused by the cache associativity. To calculate it we need to divide de 64K cache by the 8K stride, which gives us a 8-way associative cache.

3.1.2 Modeling the L2 Cache

a) Describe and justify the changes introduced in this program.

We did two changes to the program. We changed the processor event that is being analyzed to PAPI_L2_DCM because we are now studying the L2 data cache misses and changed the value of CACHE_MAX to 2MB because the L2 cache is much bigger than the L1 cache and we need arrays bigger than the L2 cache.

b) Plot the variation of the average number of misses (*Avg Misses*) with the `stride` size, for each considered dimension of the L2 cache.



c) By analyzing the obtained results:

- Determine the **size** of the L2 cache. Justify your answer.

The size of the L2 cache is 256 KB because if we look at the graph, we can see a small increase in misses between 128 KB and 256 KB but the biggest one is between 256 KB and 512 KB. This indicates that the 512 KB is the first array that is bigger than the cache.

- Determine the **block size** adopted in this cache. Justify your answer.

By looking at the 256 KB plot line we can determine that the block size is 2^6 bytes (64 B) because until that point the stride is small enough that we are getting some hits in the same block but above that point, when the plot flattens we are always missing. This means that we are always accessing different blocks

- Characterize the **associativity set size** adopted in this cache. Justify your answer.

To calculate the associativity set size we have to look at the first cache size that is bigger than the actual L2 cache. In this case we will look at the 512K one (pink plot line). If we look at it, we see a big drop in the average misses (it comes down to zero). Since we already know that our block size is 64K, we can conclude that this drop is caused by the cache associativity. To calculate it we need to divide de 512K cache by the 32K stride, which gives us a 16-way associative cache.

3.2 Profiling and Optimizing Data Cache Accesses

3.2.1 Straightforward implementation

a) What is the total amount of memory that is required to accommodate each of these matrices?

2 Bytes/int 16 × 512 × 512 = $2^1 \times 2^9 \times 2^9$ =
= 2^{19} Bytes = 512 KBytes

b) Fill the following table with the obtained data.

Total number of L1 data cache misses	135,015307	$\times 10^6$
Total number of load / store instructions completed	536,871884	$\times 10^6$
Total number of clock cycles	644,197931	$\times 10^6$
Elapsed time	0,214734	seconds

c) Evaluate the resulting L1 data cache *Hit-Rate*:

Hit Rate = $1 - \frac{\#\text{MISSES}}{\#\text{memory access}} = 1 - \frac{135,015307 \times 10^6}{536,871884 \times 10^6} =$
$\simeq 0,75 = 75\%$

3.2.2 First Optimization: Matrix transpose before multiplication [2]

a) Fill the following table with the obtained data.

Total number of L1 data cache misses	4,219341	$\times 10^6$
Total number of load / store instructions completed	536,871884	$\times 10^6$
Total number of clock cycles	518,651788	$\times 10^6$
Elapsed time	0,172884	seconds

b) Evaluate the resulting L1 data cache *Hit-Rate*:

Hit Rate = $1 - \frac{\#MISSES}{\#MEM\ ACCESS} = 1 - \frac{4,219341 \times 10^6}{536,871884 \times 10^6} \simeq 0,992$
= 99,2%

c) Fill the following table with the obtained data.

Total number of L1 data cache misses	4,488430	$\times 10^6$
Total number of load / store instructions completed	537,920763	$\times 10^6$
Total number of clock cycles	546,853474	$\times 10^6$
Elapsed time	0,182286	seconds

Comment on the obtained results when including the matrix transposition in the execution time:

When we include the matrix transposition, time increases but not by a lot because matrix transposition is less computationally intensive ($O(m^2)$) compared to matrix multiplication ($O(m^3)$).

d) Compare the obtained results with those that were obtained for the straightforward implementation, by calculating the difference of the resulting hit-rates ($\Delta\mathrm{HitRate}$) and the obtained speedups.

$\Delta\mathrm{HitRate} = \mathrm{HitRate}_{\mathrm{mm2}} - \mathrm{HitRate}_{\mathrm{mm1}}$: 99,2% - 75% = 24,2%
$\mathrm{Speedup}(\#\mathrm{Clocks}) = \#\mathrm{Clocks}_{\mathrm{mm1}}/\#\mathrm{Clocks}_{\mathrm{mm2}}$: 1,242
$\mathrm{Speedup}(\mathrm{Time}) = \mathrm{Time}_{\mathrm{mm1}}/\mathrm{Time}_{\mathrm{mm2}}$: 1,242
Comment: We obtained big differences in hit rate but relatively small speedup. This could have happened because some accesses are being cached by the L2 cache in straightforward implementation, reducing the overall miss penalty in this implementation.

3.2.3 Second Optimization: Blocked (tiled) matrix multiply [2]

a) How many matrix elements can be accommodated in each cache line?

$$\text{Elements} = \frac{\text{Block size} \times \text{Associativity}}{\text{Element size}} = \frac{2^7 \times 2^3}{2} = 2^9$$

b) Fill the following table with the obtained data.

Total number of L1 data cache misses	5,093882	$\times 10^6$
Total number of load / store instructions completed	536,888314	$\times 10^6$
Total number of clock cycles	212,007647	$\times 10^6$
Elapsed time	0,070669	seconds

c) Evaluate the resulting L1 data cache *Hit-Rate*:

$$\text{HitRate} = 1 - \frac{\#\text{Misses}}{\#\text{Mem Accesses}} = 1 - \frac{5{,}093882 \times 10^6}{536{,}888314 \times 10^6} = 0{,}991 = 99{,}1\%$$

d) Compare the obtained results with those that were obtained for the straightforward implementation, by calculating the difference of the resulting hit-rates ($\Delta\text{HitRate}$) and the obtained speedup.

$\Delta\text{HitRate} = \text{HitRate}_{\text{mm3}} - \text{HitRate}_{\text{mm1}}$:	99,1% - 75% = 24,1%
$\text{Speedup}(\#\text{Clocks}) = \#\text{Clocks}_{\text{mm1}}/\#\text{Clocks}_{\text{mm3}}$:	3,04
Comment: By using submatrices to multiply we guarantee that we have the elements we need in our cache. Using the L1 size we profit in time by using the fastest cache.	

e) Compare the obtained results with those that were obtained for the matrix transpose implementation by calculating the difference of the resulting hit-rates ($\Delta\text{HitRate}$) and the obtained speedup. If the obtained speedup is positive, but the difference of the resulting hit-rates is negative, how do you explain the performance improvement? (Hint: study the hit-rates of the L2 cache for both implementations;)

$\Delta\text{HitRate} = \text{HitRate}_{\text{mm3}} - \text{HitRate}_{\text{mm2}}$: 99,1 % − 99,2 ≈ −0,1 %
$\text{Speedup}(\#\text{Clocks}) = \#\text{Clocks}_{\text{mm2}}/\#\text{Clocks}_{\text{mm3}}$: 2,45
Comment: By using submatrices that can fit in L1 cache we have almost guaranteed that we have all the values we need in the fastest cache possible. This is why despite having a negative ΔHitRate we can have relatively big speedup.

3.2.3 Comparing results against the CPU specifications

Now that you have characterized the cache on your lab computer, you are going to compare it against the manufacturer's specification. For this you can check the device's datasheet, or make use of the command `lscpu`. Comment the results.

Intel Core i5-8500 3.00 GHz L1d : 32 K 8 Way L2 : 256 K 4 Way	All of the result obtained are the same as the specifications of the CPU with the exception of the associativity of the L2 cache, where we said it was 16-way associative. This can be due to others programs running on the PC at the same time, using up the cache.